

Gluttonous Snake: A FPGA-Based Fully Unified Accelerator for Nonlinear Functions in LLMs

Yutong Jia*, Shilin Zhang†, Xiangming Guo*, Kangping Wang‡, Chuantao Zheng§

^{*},[§]College of Electronic Science and Engineering, Jilin University, China

[†],[‡]College of Computer Science and Technology, Jilin University, China

^{*},[†]{jiayt1922,zhangsl2122,guoxm1923}@mails.jlu.edu.cn, [‡],[§]{Wangkp, zhengchuantao}@jlu.edu.cn

Abstract—Nonlinear functions constitute a major bottleneck in large language model (LLM) inference. Existing approaches often rely on isolated approximations or auxiliary modules, lacking a unified and efficient architecture. This paper presents Gluttonous Snake, a fully unified FPGA-based accelerator that integrates four critical nonlinear operations of LLMs—attention, normalization, activation, and element computation—within a single hardware framework. Our architecture employs a decomposition-reuse strategy, breaking down complex operations into fundamental operators executed via a unified on-chip network with mixed-precision arithmetic. This design enhances hardware reuse and scalability while maintaining numerical accuracy. Gluttonous Snake processes a 64x768 vector in under 6 microseconds at 300 MHz on FPGA, achieving a 300x speedup over a 3.8 GHz AMD 7840HS CPU with minimal L2 error. The code is available at https://github.com/Jsupermaster/Gluttonous_Snake.

Index Terms—hardware acceleration, nonlinear functions

I. INTRODUCTION

While the self-attention mechanism has revolutionized NLP, the computational intensity of Large Language Models (LLMs) poses significant challenges for deployment, particularly during inference. Among the key bottlenecks are nonlinear functions like Softmax and GeLU, which can consume over 50% of the execution time. While FPGA-based acceleration offers a promising path, existing approaches often suffer from a critical trade-off: they either focus on approximating a single function (e.g., Softmax) [1] with limited precision and generality, or lack dedicated, efficient hardware for a comprehensive set of essential nonlinear operations like GeLU [2]. This fragmentation highlights the need for a unified and scalable hardware architecture that can handle multiple key functions with high precision and efficiency.

In response, this paper proposes Gluttonous Snake, a unified and scalable accelerator architecture designed for high-precision computation of key nonlinear functions in LLMs. Its modular, plug-and-play design not only enhances parallel processing and resource efficiency but also ensures exceptional usability, functioning effectively both as a standalone accelerator or a co-processor. The principal contributions of this work are threefold:

- **A Unified and Scalable Acceleration Framework:** We introduce the novel Gluttonous Snake architecture, which decomposes diverse nonlinear functions into parameterized computing units connected via a configurable network. This design enables hardware reuse and allows

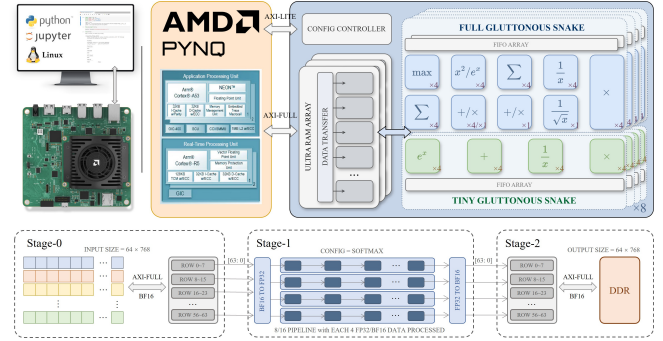


Fig. 1. Overall System Architecture Diagram

seamless integration of new functions through modular expansion.

- **An Architecture for Efficient High-Precision Computation:** We introduce a co-designed heterogeneous architecture and an intelligent hybrid-precision scheduler that collectively maximize computational throughput under resource constraints while preserving numerical stability.
- **Comprehensive System-Level Validation:** We demonstrate the practicality and generalizability of our accelerator through a system-level implementation, including a user-friendly Python interface. Extensive experiments on models like DistilGPT2, T5-Small, and RBT3 confirm its efficiency and ease of integration.

II. OVERVIEW OF THE ACCELERATION SYSTEM

This work classifies common nonlinear computations in LLMs into four categories: self-attention, normalization, activation, and element-wise operations. The accelerator features a controller, a data cache, and two processing units: eight full-function (FULL) and eight simplified (TINY) Gluttonous Snake blocks. FULL blocks process all four nonlinear operations, while TINY blocks accelerate activation and element-wise functions. This design supports 8×4-way parallelism for attention/normalization and 16×4-way for activation/element-wise computations. To balance accuracy and efficiency, FULL blocks use FP16 for reciprocal operations and FP32 for others, while TINY blocks uniformly employ BF16 precision.

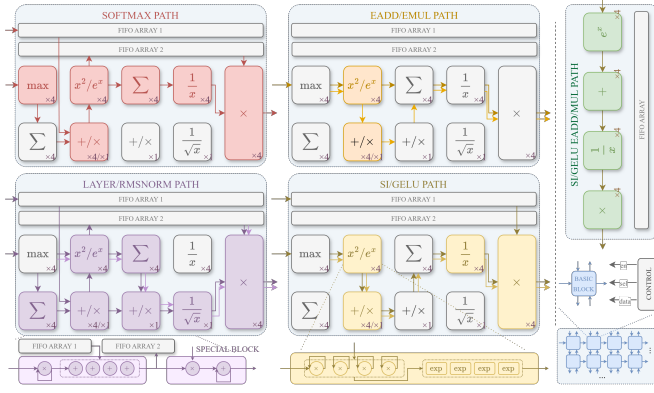


Fig. 2. System Calculation Flowchart

A. Basic Computing Unit Design

The Gluttonous Snake architecture decomposes complex nonlinear functions into elementary and specialized computing units, interconnected via a reconfigurable Network-on-Chip (NoC). The elementary units execute single operations in a continuously saturated state, while the specialized units integrate related functions (e.g., multiply-add, square-exponential) to enhance computational density and hardware reuse. Each unit features four directional data ports (West/East for input/output, South/North for bidirectional transfer). A global configuration register enables dynamic data routing: unselected units forward data packets after a two-cycle latency, while the selected unit processes data from and to its designated ports. This flexible network avoids complex gating in linear pipelines, simplifies hardware reuse design, and allows integration of new functions through reconfiguration.

B. On-chip Network Design

Figure 2 illustrates the computational workflow of the seven supported functions within the Gluttonous Snake architecture, depicting the data paths through the network in both FULL and TINY processing modes. The architecture maps seven key nonlinear functions onto four fundamental types of nonlinear operations, enabling their high-efficiency computation. Benefiting from its modular design, Gluttonous Snake possesses significant scalability and adaptability to keep pace with the rapidly evolving LLM field. The architecture can be readily reconfigured to accommodate new functions as requirements emerge, simply by updating block registers, adding or removing computational units, or modifying the network topology.

III. EXPERIMENT AND CONCLUSION

This system is implemented on an AMD KV260 platform with a Zynq UltraScale+ MPSoC (XCZU5EV) and evaluated using the PYNQ framework. The accelerator operates at 300 MHz and achieves low-latency execution of common LLM operations. For a 64×768 tensor block, key nonlinear functions complete in under $6 \mu s$ (Softmax, LayerNorm, RMSNorm) and below $3 \mu s$ (SiLU, GeLU, EAdd, EMul), with L2 errors

below 1×10^{-4} compared to the standard PyTorch implementation. The design also demonstrates significant speedup over general-purpose CPU/GPU implementations and baseline HLS-generated accelerators.

To evaluate the system's effectiveness, we deployed three distilled Language Models—DistilGPT2, T5-Small, and RBT3 [3]–[5]—and replaced their nonlinear operators with our encapsulated functions mapped to the hardware accelerator, while the remaining operations ran on the ARM CPU. This mapping reduced the execution time of key nonlinear operators by a factor of $20\times$ to $40\times$ compared to the CPU implementation.

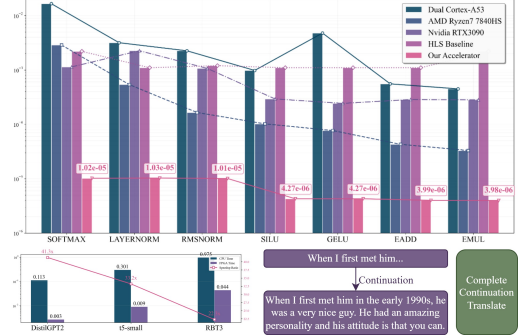


Fig. 3. Comparison of Latency on Different Platforms

TABLE I
COMPARISON OF RESULTS

Approach	Clock(MHz)	Test Data	Delay(Cycles)	Function	Resource(LUT)
Hyft16 [1]	645	1×128	18	Softmax	19026
Hyft32 [1]	526	1×128	18	Softmax	46706
ICFPT21 [2]	410	1×1	8	SiLU/GeLU	126
Ours	300	64×768	799-2050	7 Funcs ¹	104229

¹ The measured latency in cycles for each operation is as follows: Softmax (2050), LayerNorm (1873), RMSNorm (1860), SiLU/GeLU (854), and EAdd/EMul (799).

Experimental results confirm that the Gluttonous Snake array offers a systematic solution, distinct from single-function existing approaches. It concurrently supports multiple nonlinear computations and larger-scale data, with high scalability and ease of integration. The design effectively balances processing latency, accuracy, and resource use, proving its practical value.

REFERENCES

- [1] T. Xia and S. Q. Zhang, "Hyft: A reconfigurable softmax accelerator with hybrid numeric format for both training and inference," *Proceedings of the 29th ACM/IEEE International Symposium on Low Power Electronics and Design*, 2023.
- [2] X. Feng, Y. Li, Y. Qian, J. Gao, W. Cao, and L. Wang, "A high-precision flexible symmetry-aware architecture for element-wise activation functions," in *2021 International Conference on Field-Programmable Technology (ICFPT)*, pp. 1–4, 2021.
- [3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," 2020.
- [4] C. Raffel, N. M. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, pp. 140:1–140:67, 2019.
- [5] Y. Cui, W. Che, T. Liu, B. Qin, and Z. Yang, "Pre-training with whole word masking for chinese bert," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3504–3514, 2021.